

A new method for vulnerability and risk assessment of IoT

Ferhat Arat^a, Sedat Akleylek^{b,c,d,*}

^a Department of Software Engineering, Samsun University, Samsun, Turkey

^b Department of Computer Engineering, Ondokuz Mayıs University, Samsun, Turkey

^c Cyber Security and Information Technologies Research and Development Center, Ondokuz Mayıs University, Samsun, Turkey

^d University of Tartu, Tartu, Estonia

ARTICLE INFO

Keywords:

IoT security
Risk and vulnerability assessment
Cyber security
Attack graph
Threat assessment

ABSTRACT

In this paper, we propose a generic vulnerability and risk assessment method for IoT-enabled systems. The main aim is to provide risk detection and vulnerability assessment for IoT-based systems. We present three phases of risk assessment methodology: graph construction, attack path detection, and attack path filtering for high-level attack paths. We give attack path detection, risk level computing, and attack path removing procedures to validate these phases. We represent the IoT-based network as a graphical structure. Then, we construct the topology for a given IoT-based system. The smart home system is considered as a case scenario to present a realistic instance. The National Vulnerability Database (NVD), Common Vulnerability Scoring System (CVSS), and Common Vulnerability Exposures (CVE) metrics are used to assign vulnerabilities to devices. We formulate risk factors to compute risk levels for each node, attack path, and entire graph. We use the modified Depth First Algorithm (DFS) to find all attack paths for a source and target nodes. In addition, we compute risk levels using computing procedures. Further, we filter detected attack paths considering dominance level using computational metrics. We perform the simulation on a custom Python simulator considering the designed IoT-based smart home system. We compare our proposed methods with the state of the art. According to the experimental results, the proposed methods outperform existing vulnerability-based risk assessment models regarding running time complexity and operational cost.

1. Introduction

The Internet of Things (IoT) is a collection of connected systems and devices [1]. Traditional wireless technologies are used to connect and integrate the system and devices [2]. It aims to connect objects, i.e., smart devices, sensors, and RFID tags, with each other. The main paradigm of IoT technology is to provide the transfer of collected data using varying connection types. Thousands of devices establish a massive interaction for data communication to ensure sustainable, manageable, and remote-controlled systems [3]. In addition, varying domains benefit from these systems at the user or manufacturer level. Daily life has been affected by this smart and integrated environment via several usage areas such as agriculture, industry, transportation and logistics, smart homes, and medicine.

IoT is rapidly growing via evolved technological structures [4]. New devices and systems are integrating into the IoT ecosystem to reduce direct human interaction. These integrations bring together increasing device and system heterogeneity. It has simultaneously provoked security challenges, although it provides accessibility, efficiency, and convenience. Even though IoT provides varying benefits for numerous

applications and usage domains, security concerns and issues are significant challenges [5]. The increasing number of varying integrations led to vulnerability and improper actions between interconnected objects. These security issues are the result of quick manufacturing and development strategies [6]. Manufacturers prioritize functionality and diverse applicability while ignoring critical security issues. It is difficult to integrate the traditional security protocols and mechanisms into the IoT as a compact security solution due to the limited computation, limited bandwidth, power consumption, and storage characteristics of the devices [7]. Therefore, prevention and mitigation play a critical role in providing IoT and system security. Since implementing comprehensive security requirements is not a certain solution, system modeling, risk elicitation, and risk assessment actions are required. In this context, several databases and organizations, such as CVE (Common Vulnerabilities and Exposures), CWS (Common Weakness Enumeration), and NVD (National Vulnerability Database), present access points to identify and define detected device and application vulnerabilities. The presented information shows the device vulnerability definitions, risk levels, and

* Corresponding author at: University of Tartu, Tartu, Estonia.

E-mail address: sedat.akleylek@bil.omu.edu.tr (S. Akleylek).

attack vectors. Thus, vulnerability prevention and assessment methods may be developed and applied to ensure IoT security.

1.1. Motivation and contribution

A novel multi-attacker, multi-target graph-based risk assessment model was proposed in [8]. Edge devices in the IoT ecosystem were considered to address security issues. The graph-based model represented the attackers, targets, and relations. An attack path detection model and vulnerability computing metrics were presented to identify attack paths, obtain path information, and graph risk levels. Then, dominant attack paths were determined and removed to isolate edge devices in the system.

In this paper, we present a complete vulnerability and risk assessment method for IoT-based networks to address security issues. To evaluate and assess the proposed idea, we provide an IoT-based smart home scenario. Then, we give varying procedures to detect attack paths, compute risk levels, and filter the dominant attack path. The fundamental approach of our study is to provide generic vulnerability methodologies for IoT-based systems using a graph structure. The main contributions of the study are as follows:

- We propose a vulnerability graph model for the IoT-based network that represents the devices, connections, and vulnerabilities.
- For detecting possible attack paths for the given system, we use the modified Depth First Search (DFS) algorithm. The algorithm is based on DFS and calls a computing procedure inside the method. We calculate path length as hop length and generate attack paths with varying source and target devices.
- The modified DFS algorithm uses a stack data type, and it performs pop and push operations for only one node in each step. It provides to reducing of operational costs and time complexities.
- We present three phases of security and risk computing metrics: node level, attack path level, and whole graph level. Thus, we can tackle the most vulnerable devices and attack paths to assess the system risk level. We compute the risk levels as a function of node attributes such as impact score, CVSS score, and degree.
- It is difficult to manage and prevent attackers from entering systems with a large number of devices and connections. Hence, the greedy attack path filtering approach is proposed to remove dominant attack paths from the system. It provides to manage the entire system in terms of security and risk factors.
- The time complexity and operational costs of the attack path identification and path filtering procedures are reduced. According to experimental results, our proposed path detection and filtering procedures provide better running time results according to [8,9], and [10] for the varying number of nodes.

1.2. Organization

The remaining sections of the paper are structured as follows: In Section 2, we give the related works in terms of security issues and graph-based security models. In Section 3, the general structures of the proposed methodology are detailed via a case study, formulations, and varying procedures. In Section 4, the experimental environments, assessments, and comparisons are explained. We conclude the paper in Section 5.

2. Related works

In this section, we present the known security challenges and issues in the IoT (Section 2.1), and we summarize the graph-based security assessment models for these issues (Section 2.2).

2.1. Vulnerabilities in IoT devices and networks

Although huge advancements and widespread scopes provide individual and commercial advantages, systems and devices, have exposed several security vulnerabilities in the IoT. Comprehensive studies were reported by researchers to identify these vulnerabilities and provide research points to improve security mechanisms [11].

In [12], a survey to provide an overview of IoT security was given. Confidentiality, integrity, and availability (CIA) based and layer-based security issues were tackled. Various current technologies, such as machine learning (ML), artificial intelligence (AI), and blockchain, were examined in terms of addressing and assessing the security issues in the IoT. In [13], potential security threats on the IIoT-based systems were discussed to present a taxonomy to mitigate the risks of the attacks, and some of these security attacks were explained. In addition, the authors have investigated the layered architecture of the IIoT. They presented a classification for IoT security and multi-layered categorization in [14]. The IoT architecture was detailed in the layers. It was highlighted that each IoT layer has individual security vulnerabilities according to the operating protocol and device structure. In [15], a comprehensive study was presented for security issues, requirements, possible solutions, and related limitations in IoT. An extensive taxonomy was given considering the layered architecture of the IoT. The main security challenges and potential threats were analyzed from an architectural perspective. In [16], an in-depth analysis was given of the IoT ecosystem. The main scope of the study was to examine the IoT paradigm in terms of varying criteria such as security, privacy, and protocols. The platform-based security vulnerabilities were analyzed, and security solutions were examined. In [17], a vulnerability and risk computing study was proposed for IoT environments such as healthcare, commerce, and home domains. Various qualitative and quantitative scores were computed using CVSS metrics to determine IoT device vulnerability scores. In addition, they detailed device vulnerability risk scores and exploitability points. However, the proposal did not give any case scenario and risk mitigation and assessment methodology. In [18], device-level security issues were assessed using CVSS metrics. IoT and sub-domains such as healthcare, commerce, and home were considered. Each central IoT domain was tackled in terms of devices and their vulnerability scores. Identified and determined threats were classified according to importance level, such as low, medium, and high. In addition, common threat environments and related attacks were detailed for each IoT domain. As final, recommendations can be used to mitigate threats were given for each case domain. However, graph-based pathfinding and risk-filtering methodologies were not proposed to analyze IoT networks.

2.2. Graph based risk assessment models

The security issues and potential attack surfaces were highlighted in Section 2.1. IoT devices and systems are vulnerable to attacks due to their characteristics. There are various attack and vulnerability assessment studies in the literature to bring persistent and applicable solutions for security concerns in IoT. These studies provide different approaches in terms of network representation and solution techniques. The graph model is one of these representations and technical approaches. An attack graph model shows all possible attack path sequences from the source to the target. Therefore, graph-based models can offer systematic, formal, and strong directions to determine vulnerable points and help design defense mechanisms [19]. IoT network ecosystem may be represented by considering devices and their interactions. Directed and in-directed relations establish and decide network topology for graph type.

The smart home and its vulnerable architecture were tackled in [20]. These systems were considered dynamic environments due to their IoT dependencies. The proposed architecture was constructed considering the physical and communication viewpoint. Then, a smart

home system was represented as a graph structure, and layer-based, i.e., functional, physical, and communication, architecture was embraced. Various case scenarios were performed to analyze the network vulnerability, and it was validated that the intermittent availability of nodes impacts malware propagation. According to the results, the usage of mobile nodes should be considered designing attack models.

In [21], existing attack detection methods and potential threats impacts were investigated. In addition, multi-step attack detection and assessment methods were proposed for Industrial IoT (IIoT). The main perspective of the study was processing the security event data to determine secret correlations and the attackers' intentions. The proposal considers the correlation strength of the data. The alert event tool named snort was used to calculate the effective correlation between attack paths. Then, the correlation strength matrix was constructed, and the attack threat level was calculated to identify attack expectations. As a result, forward correlation strength and maximum attack probabilities were used to detect attack paths.

Due to IoT and its heterogeneous and wide-scale structure, security issues are introduced. Focusing on this viewpoint, IoT security modeling and risk assessment methods were proposed in [22]. The proposed architecture included five phases: (1) data processing, (2) model generation, (3) model visualization, (4) security assessment, and (5) model updates. System information and security metrics were determined, a security and vulnerability model was generated, potential attack paths were detected, and the impact of attacks was mitigated. The proposed framework was evaluated using varying case scenarios to monitor the model's capabilities and assess the impact of the attack paths.

A cost-effective attack detection model was proposed in [23] to address the resource-constrained structure and ensure IoT security. The predetermined all possible attack paths were detected using a greedy algorithm. Then, to provide a cost-effective security solution for IoT systems, an exploit or initial condition with a minimum effective cost metric was selected to determine the cost of the attack path. Therefore, detected attack paths were removed depending on the total cost exceeding or threshold. The experiment was performed using different scale networks, and the security level of the network was measured at each step.

The analysis of vulnerabilities for smart home-based critical systems was evaluated in [24]. In particular, critical urban environments and the known vulnerabilities of smart lighting systems were considered. CVE and CVSS metrics were used to identify vulnerabilities in the case scenario. First, interactions between the targets and devices were examined. Then, the interaction assessment phase was performed using the CVSS-based attack vectors. Next, vulnerability risks of the detected attack paths were computed considering the likelihood and businesswise perspectives.

Industrial Control System (ICS) networks have improved with the emergence of IoT and Internet-based devices. As in conventional internet networks, ICS networks are facing varying security issues. An improved vulnerability methodology was proposed in [25] to identify vulnerable devices and determine network security challenges. Network devices were represented as nodes, and vulnerabilities were assigned to nodes using the NVD, and the CVSS metrics weighted nodes. The hybrid supply chain scenario was used to validate the proposed framework.

In [26], a maximum flow-aware vulnerability assessment model was proposed. The maximum flow of the network was considered as a decision metric. Host properties and host connections were represented as nodes and edges, respectively. An attack detection procedure was presented, and attack paths were evaluated for maximum flow.

IIoT-based risk and vulnerability assessment model was proposed in [27]. Attack path detection, threat level computing, and path filtering procedures were presented to address security issues in the IIoT. A directed graph topology was constructed to represent nodes and connections. Hop count and hot spot thresholds were used to filter attack paths.

Table 1

List of some vulnerabilities in the case scenario.

CVE ID	Device	CVSS base score ^a
CVE-2019-14402	Terminal controller	3.3
CVE-2015-0739	Sensor controller	4.0
CVE-2022-4312	Mail server	5.5
CVE-2019-11561	Alarm controller	5.9
CVE-2020-9119	Smart phone	6.2
CVE-2022-47974	Bluetooth module	6.5
CVE-2018-11315	Smart thermostat	6.5
CVE-2022-28743	IP camera	6.6
CVE-2019-0690	Network devices	6.8
CVE-2018-4094	Smart TV	7.8
CVE-2018-8346	Operating dystem	8.8
CVE-2018-8045	SQL server	8.8
CVE-2018-6294	Smart cams	9.8

^a Sorted by CVSS base score.

3. The proposed idea

In this section, we present the details of the proposed idea. We also support the designed methodology using case scenarios, formulations, and procedures.

3.1. Case scenario

This section presents an example of an IoT-based smart home scenario for an easy understanding of the proposed vulnerability assessment model. Fig. 1 demonstrates the designed IoT-based smart home scenario.

The IoT is the key enabling technology due to its rapidly evolving structure and wide scope. Smart home technology is an instance of an IoT-enabled application domain. This technology provides many facilities and effectiveness to users via varying systems such as gas detection, smart lights, smart locks, temperature monitoring, and controlling [28]. Although they have numerous advantages, smart home systems consist of resource-constrained devices and have high system availability [29]. In addition, these systems share data with devices and users to ensure data communication. It causes new security and privacy issues due to the high number of connections, vulnerable devices, heterogeneous device structure, and physical accessibility.

To assess and evaluate the security of the smart home system, we benefit from varying vulnerability organizations and databases such as CVE and NVD. We also use computation metrics via CVSS. CVSS is a framework to present definition for the impact of vulnerabilities and exploits executed on a multitude of devices. Resource-constrained IoT devices consist of different vulnerabilities defined in these databases [18]. According to our assumptions, the smart home system has vulnerable and varying types of smart devices. These vulnerabilities have a base score (BS) value between 0 and 10, determining the impact on the systems and individual user actions. For instance, CVE-2018-4094 is a smart TV audio system vulnerability that causes denial of service and allows the attacker to execute arbitrary code. The base score value was 7.8 for this vulnerability, and considered a high-risk level. The CVE-2018-8045 is another vulnerability for SQL statements, leading to SQL injection vulnerability. The CVE-2018-11315 was defined as a smart thermostat vulnerability that allows unauthorized access, which has a medium risk level. Some vulnerabilities assumed in Fig. 1 are shown in Table 1.

3.2. Construction of the graph based security model

In this section, we present the general definition of the proposed graph-based vulnerability identification and assessment model. Fig. 2 describes the workflow of our model in three phases: graph construction, attack path detection, and vulnerability assessment.

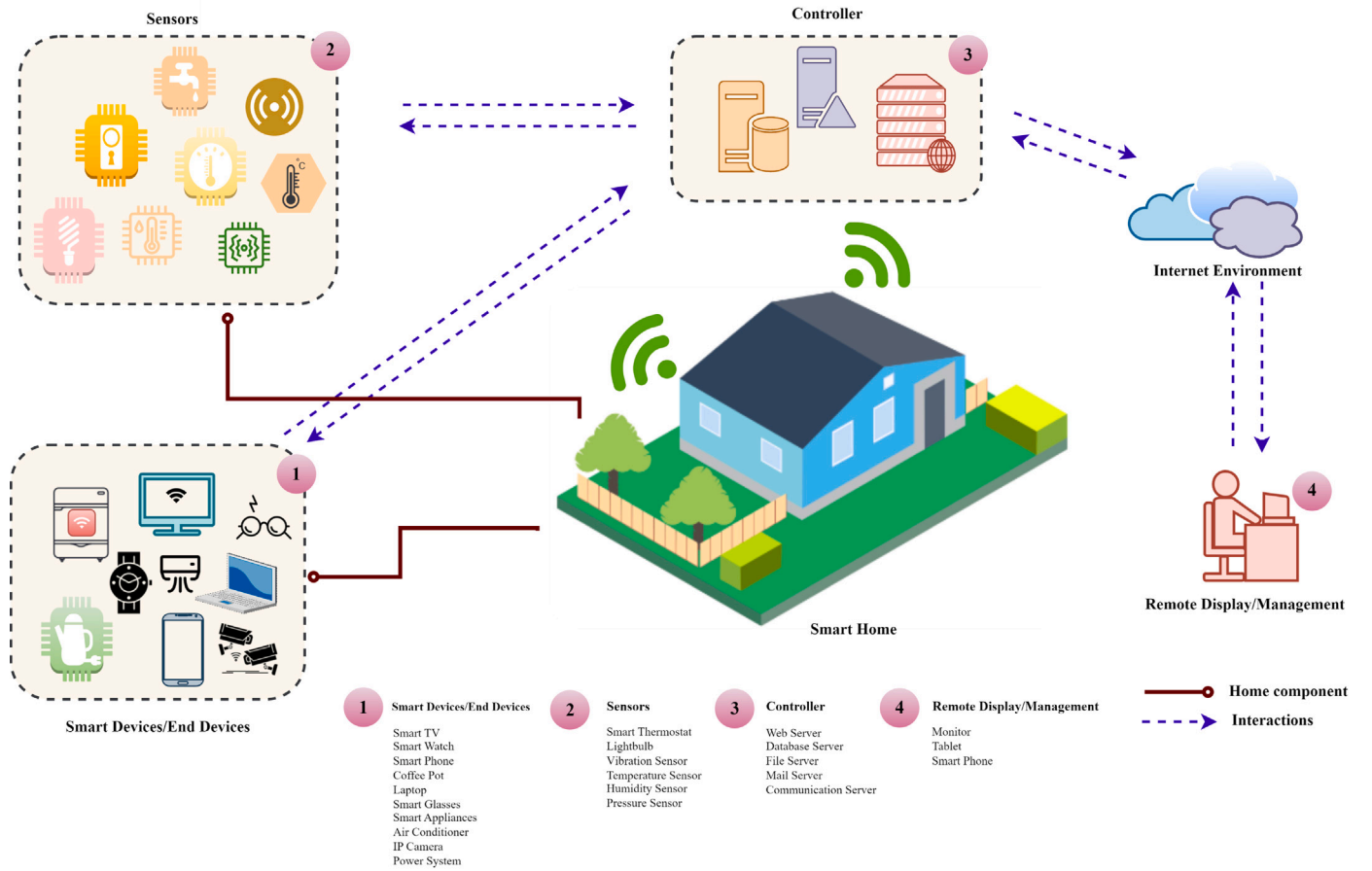


Fig. 1. An IoT-based smart home network.

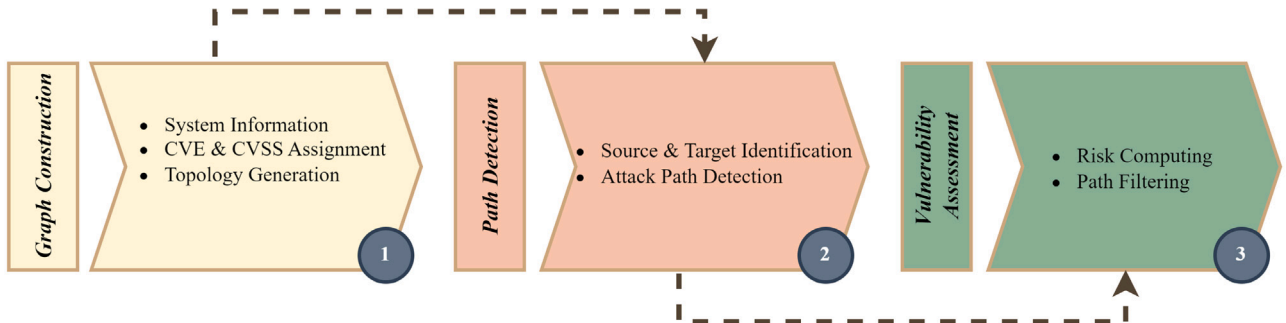


Fig. 2. Workflow for the proposed model.

- **Graph construction** phase consists of three sub-phases: system information, CVE, CVSS assignment, and topology generation. We assume a smart home scenario for the implementation. Therefore, we represent all devices as nodes, and all interaction links between nodes are represented as edges. We construct the topology using nodes and edges. In the smart home system, there are varying network and physical devices to ensure communication, and we assume that each device has a predefined vulnerability, as shown in Table 1. We assign vulnerabilities and CVSS scores to devices using CVE and CVSS.
- **Path detection** phase consists of two sub-phases: source and target identification and attack path detection. The smart home system has critical access points and devices. We assume that each node may be considered critical. Therefore, considering the interaction between nodes, we change the source and target node

pairs. Then, we find all attack paths between the source and destination pairs using the modified DFS algorithm.

- **Vulnerability assessment** phase consists of two sub-phases: risk computing and path filtering. We assign vulnerabilities and CVSS scores to the nodes and find attack paths between desired nodes in phases 1 and 2, respectively. Next, we compute attack path risk levels for each detected attack path using CVSS scores and computing metrics. Thus, we assess each attack path according to risk levels as low and high. Then, we filter the attack paths considering desired risk thresholds by the greedy algorithm.

As the main phases of the proposed model are given, we explain the formal definition of the graph model referred to as Vulnerability and Risk Graph (VRG). We define the VRG formally as follows:

$$VRG = (VRT, VI, AT, AP)$$

Definition 1 (Vulnerability and Risk Topology). Vulnerability and Risk Topology (VRT) is a collection of the device and related interactions. In the given Vulnerability and Risk Topology $VRT = (V, E)$

- $VRT \subset VRG$
- $V = (v_1, v_2, \dots, v_n)$ is the set of the nodes.
- $E = (e_1, e_2, \dots, e_n)$ is the set of edges.

Definition 2 (Vulnerability Instance). The VRG is constructed with vulnerable devices due to device attributes. Therefore, each vulnerability is an instance of a VRG. Let the VI represent a vulnerability instance for VRG. Each VI exists inside any of these devices. We define any vulnerability instance as a subset of the VRG .

- $VI \subset VRG$
- $VI_i \rightarrow \{\beta, \lambda\}$
- β is the vulnerability ID for the device.
- λ is the vulnerability score identified in CVE.

Definition 3 (Attacker Instance). An attacker is defined as a malicious user with the potential to harm the system and device by exploiting existing vulnerabilities, and AT is the attacker sub-set of the VRG .

- $AT \subset VRG$
- $AT = (a_1, a_2, \dots, a_n)$

Definition 4 (Attack Path). We assign a vulnerability and CVSS score to each node n . For any node and edge pair (v_i, e_j) , an attacker a_i can use the node n , communication link e , to reach the target, which can cause damage to the device or system. All possible attack paths P_T^{ij} for the given initial node i and the desired target node j can be represented as follows:

- P_T is the set of the attack paths,
- $P_T^{ij} = (p_1^{ij}, p_2^{ij}, \dots, p_n^{ij})$

We represent the attack paths using mentioned definitions as below:

$$AP = \begin{cases} \forall a_i \in AT & i \geq 0 \\ \forall VI_j \in VT & j \geq 0 \\ \forall (src_m \& dest_n) \in V & m, n \geq 0 \\ P^{xy} = (a_i \times VI_j) \rightarrow (src_m \times dest_n) & x, y \geq 0 \end{cases}$$

3.3. Formulation of the graph based security model

To assess the secured and unsecured attack paths and to validate the attack path filtering mechanisms, we use varying security metrics: vulnerability impact, attack success probability, and system risk level. The vulnerability and security metrics are calculated using the CVSS at three levels: node level, attack path level, and whole graph level. At the node level, we assume that each vulnerability score value determines the device's importance for the system. At the attack path level, we compute the attack path success probability and attack path risk level for the attacker. Thus, an attack path from source to target may be assessed to determine damage to the system. We evaluate the network security level for the user and attacker in the graph level.

Definition 5 (Node Level Security). We define the node level (NL) security score as a function of node vulnerability CVSS, Impact Score (IS), and probability. Each node has a vulnerability and defined metrics in NVD, and the probability value is assigned to each node according to BS. The NL is calculated in Eq. (1).

$$NL^{v_i} = CVSS_{BS}^{v_i} \times IS^{v_i} \times prob^{v_i} \quad (1)$$

where v_i is in V , and BS is calculated in NVD. The probability value $prob$ refers to the parameter defined in the range $(0,1]$, and it follows:

$$prob^{v_i} = \begin{cases} \forall prob^{v_i} < (\frac{CVSS_{BS}^{v_i}}{10}) & , 0 < prob^{v_i} \leq 1 \end{cases}$$

Definition 6 (Attack Path Level Security). We define the attack path level (APL) vulnerability score as a function of the degree of the node and NL_{node} . The degree is related to the number of direct connections in and out of the node. APL is calculated in Eq. (2), and the average APL is calculated in Eq. (3).

$$APL^{ij} = \sum NL^{v_i} \times \delta^{v_i} \quad (2)$$

$$APL_{AVG}^{ij} = \frac{NL^{v_i} \times \delta^{v_i}}{L(p^{ij})} \quad (3)$$

where L is the length of the identified attack path p_{ij} , and δ is the degree of the v_i .

We assume that the node with the highest probability in the attack path is the most vulnerable. Therefore, the vulnerability value of the attack path is equal to the highest probability value among the nodes in the attack path since an attacker can damage the system as much as the most vulnerable node in the attack path.

Definition 7 (Graph Level Security). We define graph level security (GL) as a summation function of APL. GL is related to attack path level security and node level security.

Let the list of the all attack paths AP be $P_T^{ij} = (p_1^{ij}, p_2^{ij}, \dots, p_n^{ij})$. The length of AP is n , and $n > 0$. We compute GL as in Eq. (4).

$$GL = \sum_{k=1}^n APL_k^{ij} \quad (4)$$

3.4. Vulnerability path identification

In this section, we present the details of the attack path detection model given in Algorithm 1.

Algorithm 1 Algorithm to Find Possible Attack Paths

```

1: function GETALLPATHSBYDFSgraph, src, dest, visited, path, pathList, hop
2:   visited[src] ← True
3:   Add src to table path
4:   hop ++
5:   if src = dest then
6:     COMPUTENpath                                     ▷ Compute  $NL^{v_i}$ 
7:     Add path to table pathList
8:   else
9:     for each node adjacent to src do
10:      if visited[node] ← True then
11:        GETALLPATHSBYDFSgraph, node, dest, visited, path, pathList
12:      end if
13:    end for
14:  end if
15:  Pop to last inserted node from table path
16:  path.pop()
17:  visited[src] ← False

```

Algorithm 1 is a modified form of the DFS algorithm. It takes seven inputs to initialize and runs recursively for the given source src and destination $dest$ node pairs. The procedure initializes when src is added to the temporary path object. Next, it continues until the case of $src = dest$ (Line 5). Since the algorithm uses a stack object, the operated node is removed when the visited state for the node equals *True* (Line 16). In addition, *COMPUTENpath* function is also called to compute node level security NL^{v_i} for each visited node (Line 6).

3.4.1. Complexity analysis for Algorithm 1

In this section, we present the complexity analysis for Algorithm 1.

Theorem 1. Algorithm 1 runs in $\mathcal{O}(V + E)$.

Proof. Algorithm `GETALLPATHSByDFSgraph`, `src`, `dest`, `visited`, `path`, `pathList` runs on the given $VRT = (V, E)$, where the V is the set of the nodes and the E is the set of the edges. The algorithm finds the attack paths using the given `src` and `dest` as parameter pairs. In the given topology VRT , the modified DFS algorithm traverses all nodes recursively in V using connections, and it takes the complexity of $\mathcal{O}(V + E)$ (In lines 2–13). Since the DFS algorithm uses stack data type, popping and inserting an element to the stack takes $\mathcal{O}(1)$, and for the n number of nodes, it takes $\mathcal{O}(1)$ complexity. The final complexity of the given Algorithm 1 is $\mathcal{O}(V + E)$.

3.5. Risk computing and path filtering by greedy approach

In this section, we present the details of the attack path filtering models given in Algorithms 2, 3, and 4.

Algorithm 2 Algorithm to Compute Node Level Risk

```

1: function COMPUTENgraph, pathList
2:  $res \leftarrow 0$ 
3:  $newList \leftarrow \emptyset$ 
4: for each  $p$  in  $pathList$  do
5:    $res \leftarrow NL^{U_i}$  ▷ use the Eq. (1)
6:    $newList \leftarrow p$ 
7: end for
8:  $newList \leftarrow res$  ▷ Appends as last index

```

Algorithm 2 takes one input to compute node-level security. It uses the returned parameter from Algorithm 1 to compute the risk level for each node (Line 4). It assigns the computed value to the temporary variable and stores the risk value as the last index of the `newList`.

Algorithm 3 Algorithm to Compute Path Level Risk

```

1: function COMPUTEPgraph, completeList
2:  $cumulativeThreat, avgThreat \leftarrow 0$ 
3: for each  $subList$  in  $completeList$  do
4:    $cumulativeThreat \leftarrow subList[-1]$ 
5: end for
6:  $avgThreat = cumulativeThreat / \text{length of } completeList$ 
7: FILTERcompleteList, avgThreat

```

Algorithm 3 takes two inputs. The procedure assigns the initial values to the parameters (Line 2). Then, the cumulative attack path threat value referred to as APL^{ij} is computed. As seen in Line 4, we use the list data type to store the calculated risk value as the last element. Therefore, we access the computed value without any temporary variables. The `FILTERgraph`, `completeList`, `riskThreshold` function is called when computing of path level risk value APL^{ij} is over.

Algorithm 4 Algorithm to Filter Attack Paths using Greedy Approach

```

1: function FILTERgraph, completeList, riskThreshold
2:  $filteredList \leftarrow \emptyset$ 
3: for each  $subList$  in  $completeList$  do
4:   if  $subList[-1] \leq riskThreshold$  then
5:      $filteredList \leftarrow subList$ 
6:   end if
7: end for

```

Algorithm 4 takes three inputs. It filters the detected attack paths considering the given threshold metric. We use the average value of the GL to determine risk threshold. The risk threshold value is determined using the computed GL . The procedure iterates until the length of the all-attack path list (Line 3). Then, it filters attack paths by comparing the last element of path lists with the risk threshold value (Lines 4–5).

Table 2

The initial parameters for the nodes.

Nodes	$node_0$	$node_1$	$node_2$	$node_3$	$node_4$	$node_5$	$node_6$	$node_7$	$node_8$	$node_9$
CVSS	4.7	5.5	3.8	4.6	2.1	6.0	7.7	9.1	8.5	5.9
IS	3.2	3.4	2.9	3.1	2.7	4.1	4.9	6.8	5.1	3.6
prob	0.27	0.3	0.2	0.28	0.1	0.37	0.42	0.78	0.64	0.35
δ	1	2	2	2	3	2	3	2	2	3

3.5.1. Complexity analysis for Algorithms 2, 3, and 4

In this section, we present the complexity analysis for Algorithm 3, and Algorithm 4.

Theorem 2. Algorithm 2 runs in $\mathcal{O}(n)$.

Proof. The node level risk computing procedure given in Algorithm 2 assigns the initializing values to the variables, and it takes constant time $\mathcal{O}(1)$. The `pathList` variable stores the detected attack paths as a list data type. Let the length of the attack path be n . Then, the procedure iterates for n times. It takes linear time $\mathcal{O}(n)$. The time complexity of the given algorithm is $\mathcal{O}(n)$.

Theorem 3. Algorithm 3 runs in $\mathcal{O}(n)$.

Proof. The path level risk computing procedure given in Algorithm 3 assigns the initializing values to the variables, and it takes constant time $\mathcal{O}(1)$. The `completeList` variable stores the detected attack paths as a list data type. Let the length of the attack path be n . Then, the procedure iterates for n times. It takes linear time $\mathcal{O}(n)$. The cumulative threat value is computed by accessing the last element of the attack path list. Accessing an element of a list takes $\mathcal{O}(1)$. Finally, the procedure computes the graph's average threat, which takes constant time. Therefore, the time complexity of the given procedure is $\mathcal{O}(n)$.

Theorem 4. Algorithm 4 runs in $\mathcal{O}(n)$.

Proof. The filtering procedure given in Algorithm 4 assigns the initializing values to the variables, and it takes constant time $\mathcal{O}(1)$. Next, it iterates for the length of the given list `completeList` and takes $\mathcal{O}(n)$. Therefore, the final time complexity of the procedure is $\mathcal{O}(n)$.

4. Experimental results

In this section, we give the experimental details and experimental results for the proposed procedures. In addition, we run the procedures using the example topology, and the proposed models are compared with [8]

4.1. Experimental setup

This section gives an example topology to perform the proposed procedures and summarizes the results. Fig. 3 shows the example topology for the given scenario.

An example scenario given in Fig. 3 consists of 10 vulnerable nodes. Each node represents a device in the case scenario. We assume that the initial node degree is 1, and $node_0$ and $node_9$ represent the source and destination nodes, respectively. Table 2 illustrates the initial parameters for the nodes.

Table 3 illustrates the results of the attack path detection procedure given in Algorithms 1 and 3.

Table 4 stands for the results of the node-level security computing procedure given in Algorithm 2.

Table 5 illustrates the outputs for Algorithm 4. As shown in the table, detected attack paths are filtered using the procedure for the given source and destination node pair. The average risk threshold value is considered a filtering metric.

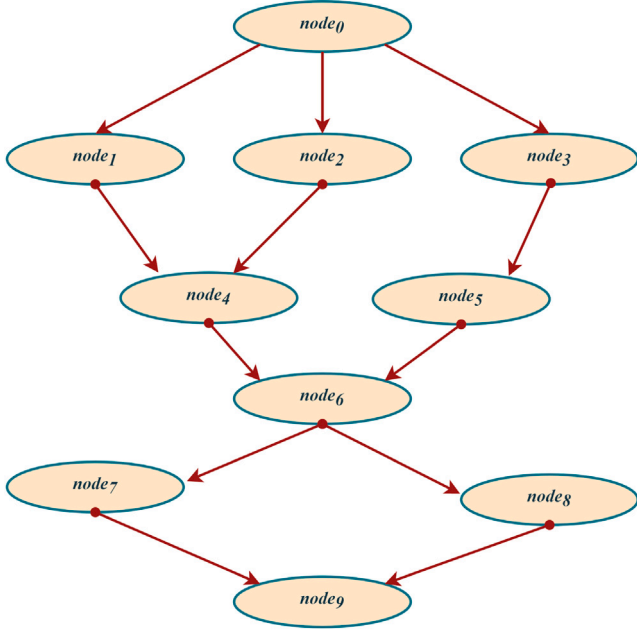


Fig. 3. A vulnerability graph example of the given scenario.

Table 3

Results for Algorithm 1.

Path No	Path sequence	$APL^{0.9}$	$APL_{AVG}^{0.9}$
1	0 1 4 6 7 9	168.43	33.68
2	0 1 4 6 8 9	127.39	25.47
3	0 2 4 6 7 9	157.21	31.44
4	0 2 4 6 8 9	116.17	23.23
5	0 3 5 6 7 9	140.97	28.19
6	0 3 5 6 8 9	182.01	36.40

Table 4

Results for Algorithm 2.

Nodes	$node_0$	$node_1$	$node_2$	$node_3$	$node_4$	$node_5$	$node_6$	$node_7$	$node_8$	$node_9$
NL^V	4.06	5.61	2.20	3.99	0.56	9.10	15.84	48.26	27.74	7.43

Table 5

Results for Algorithm 4.

Risk threshold	148.69
GL	892.14
Removed path (s)	0 1 4 6 7 9 2 2 4 6 7 9 0 3 5 6 8 9
Remaining path (s)	0 1 4 6 8 9 0 2 4 6 8 9 0 3 5 6 7 9

4.2. Comparison

In this subsection, we compare our study with [8] in terms of algorithmic approaches and complexity. Table 6 highlights general differences between studies. According to the comparison, we can summarize the general differences between our proposed procedures as follows:

- We use the modified form of the DFS algorithm to find vulnerable paths for a given source and destination pair. The proposed method detects possible attack paths recursively using the stack data type. Therefore, it takes $\mathcal{O}(V + E)$ time.

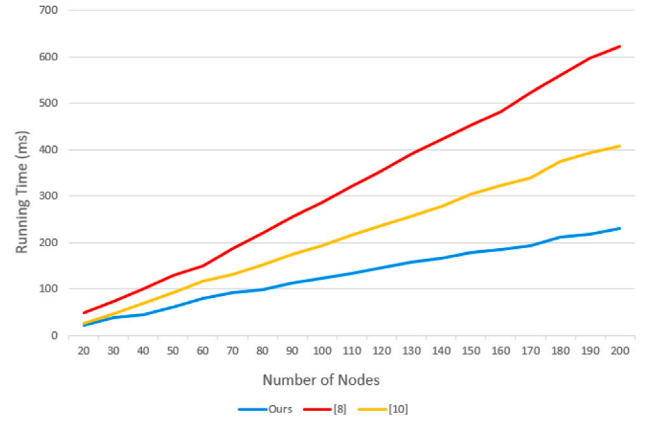


Fig. 4. Comparison of attack path finding algorithm performances with the different number of nodes.

- Our proposed risk computing procedures use a single list to store attack paths and compute risk levels. Therefore, they work linearly in terms of accessing and appending an element.
- The proposed computing and filtering procedures do not require additional storing variables using a one-dimensional array data type.
- We compute the amount of threat in three levels. It provides to obtain node level, path level, and graph level security levels.
- Our greedy filtering approach takes $\mathcal{O}(n)$ time depending on the length of the complete attack path list.

Remark 1. We use the list data type on Algorithm 4 to filter dominant attack paths. Then, we store risk threshold value as an element of the list and it is the last element of the list. Therefore, we access the any of element of list (array) in $\mathcal{O}(1)$ time complexity. In Algorithm 4, we use the *completeList* and the algorithm iterates for each paths on the list. It compares the threshold value with the last element of the attack path. It is an greedy approach in terms of all the list elements are traversed. We filter the dominant attack paths using threshold. We reduce the complexity due to used data type.

4.3. Experimental results

In this section, we present the simulation results and performance evaluations. To investigate the behaviors of proposed methods under various parameters, we change the number of nodes, the number of edges, and the threat threshold. Therefore, the proposed algorithms run over various scalable network sizes, and we present a comprehensive performance analysis. The simulation is performed on the same machine with an Intel Core i5 10300H CPU, 16 GB of RAM, and a GTX 1650 GPU. The simulation results are presented considering average values of 100 iterations.

Fig. 4 illustrates the running time comparison results between [8, 10], and our studies in terms of vulnerable path-finding algorithms. According to the results, our proposed idea performs better regarding running time under the varying number of nodes. Our proposed procedure is a modified form of the DFS algorithm. It works recursively and uses the stack data type. The procedure visits only a node for each iteration and performs push and pop operations in $\mathcal{O}(1)$ time. In [8], list data type was used, and the length and size of the list were increased for each loop. Similarly, in [10], a two-dimensional list data type stores nodes and chains. Our proposed method does not require additional storage variables and calls the computing procedure inside the algorithm. As seen in the figure, total running time increases depending on increasing the number of nodes. Since the number of

Table 6
Comparison of the algorithms.

	Ours	[8]
Algorithm to find attack paths	The algorithm performs recursively, and the content of the path list decreases for each loop because of stack object. The time complexity is $\mathcal{O}(V \times V)$ where V is the number of vertices. The auxiliary space is $\mathcal{O}(V \times V)$ where V is the number of vertices. Parameters were stored by single path list	The algorithm performs recursively, and the content of the path list increases for each loop. The time complexity is $\mathcal{O}(N_{ij} \cdot L_{ij})$ where N_{ij} is the number of paths and L_{ij} the hop-length of the longest path. General table T was used to store all parameters.
Algorithm to compute risk level	The algorithm performs in linear time. The time complexity is $\mathcal{O}(P_{ij})$ where P_{ij} is the number of paths. The hop length is generated using by path length. Computing results are stored in the same list as last element. The time complexities of the hop and risk level results are $\mathcal{O}(1)$ and $\mathcal{O}(n)$, respectively.	The algorithm performs in linear time. The time complexity is $\mathcal{O}(N_{ij})$ where N_{ij} is the number of paths. The hop length variable was generated using by different variable.
Algorithm to filter attack paths	The algorithm performs in linear time. The algorithm performs two sub-procedures. Cumulative risk threshold is determined considering the risk level of all graphs. Filtered sub-attack path list is generated using a single array data type. The time complexities of the algorithms is $\mathcal{O}(n)$ where n is the number of elements in path list.	The algorithm performs in quadratic time. The time complexity is $\mathcal{O}(n^2)$ where n is the number of nodes. The algorithm works using a greedy approach and visits all nodes in the path list.

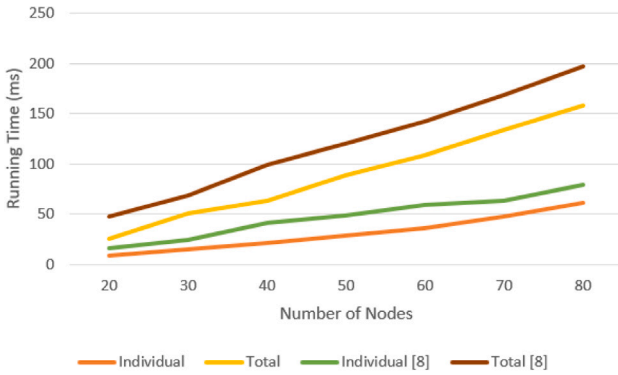


Fig. 5. Comparison of attack path filtering algorithm performances with the different number of nodes.

paths increases depending on the number of nodes, each node affects visiting possibility, and the algorithm considers the intermediate node traversing the graph. Therefore, the storage and time complexities of the algorithms increase.

Fig. 5 illustrates the running time comparison results between [8] and our studies regarding vulnerable path-filtering algorithms. The figure provides two perspectives. First, it shows the individual performances of the studies. As shown in the figure, our greedy filtering approach performs better due to the use of single-list data types. It provides visits to all paths for a single loop. In addition, our greedy approach compares the detected paths, considering the last index of the same list. The seconds show the total performances of the studies. The figure shows that our computing and filtering procedures perform better due to accessing and comparing performances. We store the computational results in the same list as the last index. Therefore, it takes $\mathcal{O}(1)$ time to access an element. It provides decreasing in operational complexities.

Fig. 6 illustrates the attack path classification according to risk level. We use the defined risk level ranges by CVE as low, medium, high, and critical. We determine the number of nodes and the number of edges as 40 and 70, respectively. In this classification, we assess the detected attack paths in terms of level to provide a perspective for the attacker and system administrators.

Fig. 7 shows the total performance of attack path filtering procedures. As shown in the figure, our proposed filtering procedure outperforms existing ones. This is because we use the array data type to store node and risk levels. It provides us access to the last element in the array in $\mathcal{O}(1)$ time complexity. We compute risk levels of the

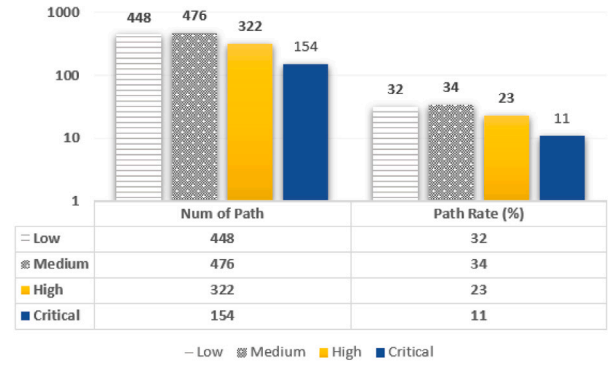


Fig. 6. Classification of detected attack paths according to risk level.

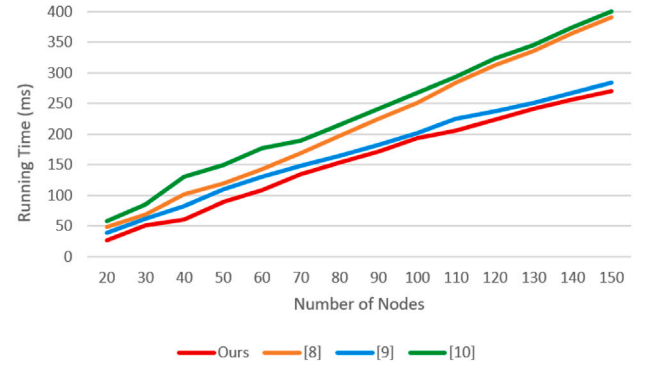


Fig. 7. Comparison of attack path filtering algorithm performances according to total running time with the different number of nodes.

node and paths; therefore, we access the computed data without any of additional data type. We store the computed data as the last element of the array. Although it is a greedy approach, we access the last element of the list in time complexity of $\mathcal{O}(n^2)$. In addition, our path and node level risk computing equations run in time complexity in $\mathcal{O}(n)$. In [9], the risk computing procedure works in time complexity in $\mathcal{O}(n^3)$. As final, our risk computing procedure reduces the time complexity of the total attack path filtering algorithm.

5. Conclusion

In this paper, we propose a vulnerability detection and assessment methodology for a given IoT-based system. We represent the IoT-based

network as a directed graph. We present a three-phase system: path-finding, risk computing, and path filtering. Firstly, we identify the attack paths using a modified DFS algorithm for a given source and destination pair. Therefore, we detect the available vulnerable paths. Then, we compute risk levels at the node level, path level, and topology level. We define the computing metrics considering the node vulnerability scores and related parameters such as node degree and impact score. As a final, we filter the detected attack paths using computed risk values to help security administrators to assess risks for the systems. We assume that the weakest device is the most vulnerable point for the system. In this context, we determine the most critical paths via the proposed methods, considering risk levels. Another significant aspect is that we reduce the complexity of risk computing and path filtering procedures. We compare the proposed methods with the literature, and the experimental results show that the proposed procedures outperform the literature regarding the time complexity and operational cost.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This study was partially supported by ASELSAN, Turkey.

References

- [1] M. Shirafkan, A. Shahidinejad, M. Ghobaei-Arani, An intrusion detection system using deep cellular learning automata and semantic hierarchy for enhancing RPL protocol security, *Cluster Comput.* (2022) 1–19.
- [2] D.D. Onthoni, P.K. Sahoo, M. Atiquzzaman, ASAA: Multi-hop and multi-user channel hopping protocols for cognitive radio enabled internet of things, *IEEE Internet Things J.* (2022).
- [3] Y. Alsaawy, A. Alkhodre, A. Abi Sen, A. Alshantiti, W.A. Bhat, N.M. Bahbouh, A comprehensive and effective framework for traffic congestion problem based on the integration of IoT and data analytics, *Appl. Sci.* 12 (4) (2022) 2043.
- [4] L. Da Xu, W. He, S. Li, Internet of things in industries: A survey, *IEEE Trans. Ind. Inform.* 10 (4) (2014) 2233–2243.
- [5] E. Schiller, A. Aidoo, J. Fuhrer, J. Stahl, M. Zörjen, B. Stiller, Landscape of IoT security, *Comp. Sci. Rev.* 44 (2022) 100467.
- [6] F. Hashmat, S.G. Abbas, S. Hina, G.A. Shah, T. Bakhshi, W. Abbas, An automated context-aware IoT vulnerability assessment rule-set generator, *Comput. Commun.* 186 (2022) 133–152.
- [7] M. Zahid, I. Inayat, M. Daneva, Z. Mehmood, Security risks in cyber physical systems—A systematic mapping study, *J. Softw.: Evol. Process* 33 (9) (2021) e2346.
- [8] G. George, S.M. Thampi, Vulnerability-based risk assessment and mitigation strategies for edge devices in the internet of things, *Pervasive Mob. Comput.* 59 (2019) 101068.
- [9] F. Arat, S. Akleyek, Attack path detection for IIoT enabled cyber physical systems: Revisited, *Comput. Secur.* 128 (2023) 103174.
- [10] I. Stelios, P. Kotzanikolaou, C. Grigoriadis, Assessing IoT enabled cyber-physical attack paths against critical systems, *Comput. Secur.* 107 (2021) 102316.
- [11] J. Sengupta, S. Ruj, S.D. Bit, A comprehensive survey on attacks, security issues and blockchain solutions for IoT and IIoT, *J. Netw. Comput. Appl.* 149 (2020) 102481.
- [12] B.K. Mohanta, D. Jena, U. Satapathy, S. Patnaik, Survey on IoT security: Challenges and solution using machine learning, artificial intelligence and blockchain technology, *Internet of Things* 11 (2020) 100227.
- [13] A.C. Panchal, V.M. Khadse, P.N. Mahalle, Security issues in IIoT: A comprehensive survey of attacks on IIoT and its countermeasures, in: 2018 IEEE Global Conference on Wireless Computing and Networking (GCWCN), IEEE, 2018, pp. 124–130.
- [14] H. Mrabet, S. Belguith, A. Alhomoud, A. Jemai, A survey of IoT security based on a layered architecture of sensing and data analysis, *Sensors* 20 (13) (2020) 3625.
- [15] H. HaddadPajouh, A. Dehghantanha, R.M. Parizi, M. Aledhari, H. Karimipour, A survey on internet of things security: Requirements, challenges, and solutions, *Internet of Things* 14 (2021) 100129.
- [16] L. Babun, K. Denney, Z.B. Celik, P. McDaniel, A.S. Uluagac, A survey on IoT platforms: Communication, security, and privacy perspectives, *Comput. Netw.* 192 (2021) 108040.
- [17] S. Rizvi, N. McIntyre, J. Ryoo, Computing security scores for IoT device vulnerabilities, in: 2019 International Conference on Software Security and Assurance (ICSSA), IEEE, 2019, pp. 52–59.
- [18] S. Rizvi, R. Pipetti, N. McIntyre, J. Todd, I. Williams, Threat model for securing internet of things (IoT) network at device-level, *Internet of Things* 11 (2020) 100240.
- [19] J.B. Hong, D.S. Kim, C.-J. Chung, D. Huang, A survey on the usability and practical applications of graphical security models, *Comp. Sci. Rev.* 26 (2017) 1–16.
- [20] G. Kavallieratos, N. Chowdhury, S. Katsikas, V. Gkioulos, S. Wolthusen, Threat analysis for smart homes, *Future Internet* 11 (10) (2019) 207.
- [21] Z. Ding, Y. Wang, Multi-step attack threat recognition algorithm based on attribute association in internet of things security, *Wirel. Netw.* (2022) 1–12.
- [22] M. Ge, J.B. Hong, W. Guttmann, D.S. Kim, A framework for automating security analysis of the internet of things, *J. Netw. Comput. Appl.* 83 (2017) 12–27.
- [23] B. Yigit, G. Gür, F. Alagöz, B. Tellenbach, Cost-aware securing of IoT systems using attack graphs, *Ad Hoc Netw.* 86 (2019) 23–35.
- [24] I. Stelios, K. Mokos, P. Kotzanikolaou, Assessing smart light enabled cyber-physical attack paths on urban infrastructures and services, *Connect. Sci.* 34 (1) (2022) 1401–1429.
- [25] A. Ur-Rehman, I. Gondal, J. Kamruzzaman, A. Jolfaei, Vulnerability modelling for hybrid industrial control system networks, *J. Grid Comput.* 18 (4) (2020) 863–878.
- [26] H. Wang, Z. Chen, J. Zhao, X. Di, D. Liu, A vulnerability assessment method in industrial internet of things based on attack graph and maximum flow, *IEEE Access* 6 (2018) 8599–8609, <http://dx.doi.org/10.1109/ACCESS.2018.2805690>.
- [27] G. George, S.M. Thampi, A graph-based security framework for securing industrial IoT networks from vulnerability exploitations, *IEEE Access* 6 (2018) 43586–43601.
- [28] H. Touqeer, S. Zaman, R. Amin, M. Hussain, F. Al-Turjman, M. Bilal, Smart home security: challenges, issues and solutions at different IoT layers, *J. Supercomput.* 77 (12) (2021) 14053–14089.
- [29] H. Lin, N.W. Bergmann, IoT privacy and security challenges for smart home environments, *Information* 7 (3) (2016) 44.



Ferhat Arat received the B.Sc. degree in computer engineering from Ondokuz Mayıs University, Samsun, in 2018, and the M. Sc. degree in computer engineering from Ondokuz Mayıs University, Samsun, in 2021, where he is currently pursuing the Ph.D. degree in computational sciences. His research interests include the IoT, IoT security, computer networks, data communication, sensor networks, CRN.



Sedat Akleyek received the B.Sc. degree in mathematics majored in computer science from Ege University, Izmir, Turkey, in 2004, and the M.Sc. and Ph.D. degrees in cryptography from Middle East Technical University, Ankara, Turkey, in 2008 and 2010, respectively. He was a Post-doctoral Researcher at the Cryptography and Computer Algebra Group, TU Darmstadt, Germany, between 2014 and 2015. He was an Associate Professor at the Department of Computer Engineering, Ondokuz Mayıs University, Samsun, Turkey, between 2016 and 2022. He has been Professor at the Department of Computer Engineering, Ondokuz Mayıs University, Samsun, Turkey, in 2022. He has started to work at the Chair of Security and Theoretical Computer Science, University of Tartu, Tartu, Estonia since 2022. His research interests include the areas of post-quantum cryptography, algorithms and complexity, architectures for computations in finite fields, applied cryptography for cyber security, malware analysis, IoT security, and avionics cyber security. He is a member of the Editorial Board of IEEE Access, Turkish Journal of Electrical Engineering and Computer Sciences, PeerJ Computer Science, and International Journal of Information Security Science.